This works, but it's tedious. Remember, a mixin is a collection of related functions that can be attached as methods to objects. Suppose you have a collection of three functions:

```
dwarves =
  bashful: -> 'Bashful'
  doc: -> 'Doc'
  dopey: -> 'Dopey'
```

To mix dwarves into a `FairyTale` class, you add *all* of the functions to the class prototype:

```
class FairyTale
  for key, value of dwarves
    FairyTale::[key] = value
```

Comprehension of the properties of the dwarves object, adding them to the FairyTale prototype

You want a general solution, though, that will work for `Camera` and looks like so:

```
class Camera
  include @, htmlRenderer
# ReferenceError: include is not defined
```

This is the syntax you want.

You just haven't implemented it yet.

How can you make that work? The `include` function should add all of the properties from the mixin to the class:

```
x1 = new Camera
x1.render()
#renders
```

render method mixed in.

That `include` function is implemented using a comprehension:

```
include = (klass, module) ->
  for key, value of module
    klass::[key] = value
```

This include function copies all the properties from the module argument to the prototype of the klass argument.

Some people prefer to think in terms of classes. A `Mixin` class, from which you can create instances that provide their own `include` method, is shown here.

Listing 5.11. `Mixin` class